

Comunicación entre Procesos (IPC)

Sistemas Operativos
DC - FCEN - UBA

20 de Agosto de 2026

Temario

- 1 Repaso
- 2 Ejercicio 1: Los Simonzón
- 3 Ejercicio 2: Fantasma en el Shell
- 4 Ejercicio 3: ¿Cien cabezas piensan mejor que una?
- 5 Sockets
- 6 Conclusión

Temario

- 1 Repaso
- 2 Ejercicio 1: Los Simonzón
- 3 Ejercicio 2: Fantasma en el Shell
- 4 Ejercicio 3: ¿Cien cabezas piensan mejor que una?
- 5 Sockets
- 6 Conclusión

Procesos

- `int fork()`: Crea un nuevo proceso, copiando el actual. Retorna **0** en el proceso hijo, y el **PID** del proceso creado en el proceso padre.
- `void exit(int status)`: Termina el proceso actual utilizando valor de **status** como valor de retorno.

Pipes

- `int pipe(int descriptores[2]):` Crea **un pipe unidireccional**, el cual tiene un **extremo de escritura**^a, y un **extremo de lectura**^b. Genera dos descriptores, representando a los extremos de lectura y escritura respectivamente, y los guarda en **descriptores**.
- `int dup2(int oldfd, int newfd):` si `oldfd` y `newfd` son distintos, primero se elimina la referencia al objeto apuntado por `newfd`, y luego se apunta `newfd` al mismo objeto que `oldfd`.

^aPor donde “entra” la información.

^bPor donde “sale” la información.

Archivos / Descriptores

- `int open(char* path, int flags, ...)`: Abre el archivo indicado por `path`, retornando un *descriptor* que apunta a dicho archivo.
- `int close(int d)`: Cierra *para el proceso actual* el descriptor `d` pasado por parámetro.
- `int read(int d, void *b, size_t s)`: Lee `s` bytes del archivo apuntado por el descriptor `d`, y los escribe en el buffer `b`.
- `int write(int d, void *b, size_t s)`: Lee `s` bytes del buffer `b`, y los escribe en el archivo apuntado por el descriptor `d`.

Ejecutar un archivo

- `int execlp(const char *file, const char *arg, ...)`:
Sustituye la imagen de memoria del programa por la del programa ubicado en `file`. El `const char *arg` y la subsiguiente elipsis se pueden pensar como los argumentos `arg0`, `arg1`, ..., `argn` del programa a ejecutar. Por convención, `arg0` debería ser el nombre del binario (`file`). La lista de argumentos debe ser terminada con `NULL` ya que es una función variádica. Ver familia de funciones `exec`^a.

^a<https://linux.die.net/man/3/execlp>

Temario

- 1 Repaso
- 2 Ejercicio 1: Los Simonzón
- 3 Ejercicio 2: Fantasma en el Shell
- 4 Ejercicio 3: ¿Cien cabezas piensan mejor que una?
- 5 Sockets
- 6 Conclusión

Ejercicio 1: Los Simonzón



Dados y Doppelgänger...

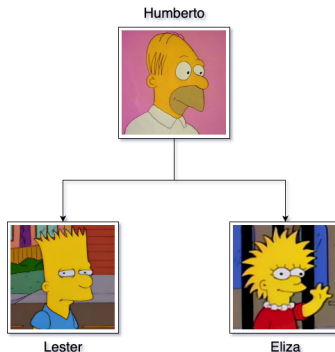
Lester y Eliza están muy aburridos. Para matar el tiempo, su ocurrente padre Humberto les propone un juego:

- Lester y Eliza tiran cada uno un dado, al mismo tiempo.
- Luego le cuentan a Humberto qué valor les dio el dado.
- Finalmente, Humberto se fija cuál es el número más grande y grita bien fuerte el nombre del ganador.

Ejercicio 1: Los Simonzón

Proponer un código en C que represente este juego respetando:

- Cada personaje debe estar representado por un proceso.
- Se debe respetar la genealogía.



- La comunicación entre procesos deberá realizarse con pipes.
- Los anuncios deberán realizarse imprimiendo a `stdout`.

Ejercicio 1: Los Simonzón

Pseudocódigo

Humberto:

Ejercicio 1: Los Simonzón

Pseudocódigo

Humberto:

- Crear a Lester.
- Crear a Eliza.

Lester:

Eliza:

Ejercicio 1: Los Simonzón

Pseudocódigo

Humberto:

- Crear a Lester.
- Crear a Eliza.

Lester:

- Tirar el dado.
- Enviar resultado a Humberto.

Eliza:

Ejercicio 1: Los Simonzón

Pseudocódigo

Humberto:

- Crear a Lester.
- Crear a Eliza.

Lester:

- Tirar el dado.
- Enviar resultado a Humberto.

Eliza:

- Tirar el dado.
- Enviar resultado a Humberto.

Ejercicio 1: Los Simonzón

Pseudocódigo

Humberto:

- Crear a Lester.
- Crear a Eliza.
- Recibir número de Lester.
- Recibir número de Eliza.

Lester:

- Tirar el dado.
- Enviar resultado a Humberto.

Eliza:

- Tirar el dado.
- Enviar resultado a Humberto.

Ejercicio 1: Los Simonzón

Pseudocódigo

Humberto:

- Crear a Lester.
- Crear a Eliza.
- Recibir número de Lester.
- Recibir número de Eliza.
- Anunciar al ganador.

Lester:

- Tirar el dado.
- Enviar resultado a Humberto.

Eliza:

- Tirar el dado.
- Enviar resultado a Humberto.

Ejercicio 1: Los Simonzón

Resolución

Nos tomamos unos minutos para pensarlo...



**Looking at
programming
memes**



**Actually
coding**

...y luego lo resolvemos en vivo.

Ejercicio 1: Los Simonzón

Solución

Importante

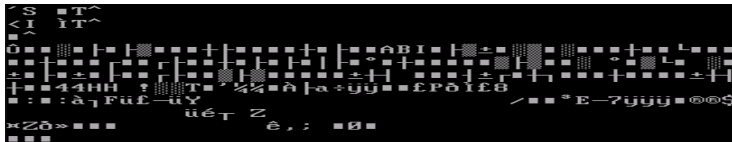
Todas las soluciones mostradas están simplificadas a efecto de poder incluirlas en la clase. Deberán tener **cuidado** y criterio a la hora de decidir si aplicar estas simplificaciones en sus resoluciones.

Es imprescindible que **lean minuciosamente el manual** para cada función que utilicen.

Temario

- 1 Repaso
- 2 Ejercicio 1: Los Simonzón
- 3 Ejercicio 2: Fantasma en el Shell**
- 4 Ejercicio 3: ¿Cien cabezas piensan mejor que una?
- 5 Sockets
- 6 Conclusión

Ejercicio 2: Fantasma en el Shell



¡No se que toqué... y ahora no anda más el shell!

Ejercicio 18 de la Práctica 1: "Procesos y APIs del SO"

Escribir el código de un programa que se comporte de la misma manera que la ejecución del comando `'ls -al | wc -l'` en un intérprete de comandos.

No está permitido utilizar la función `system()`, y cada uno de los programas involucrados en la ejecución del comando deberá ejecutarse como un subproceso.

Ejercicio 2: Fantasma en el Shell

Consideraciones preliminares

Primero, veamos qué hacen los comandos:

<code>ls -al</code>	
<code>wc -l</code>	
<code>cmd1 cmd2</code>	
<code>ls -al wc -l</code>	

Ejercicio 2: Fantasma en el Shell

Consideraciones preliminares

Primero, veamos qué hacen los comandos:

<code>ls -al</code>	Lista los archivos del directorio actual.
<code>wc -l</code>	
<code>cmd1 cmd2</code>	
<code>ls -al wc -l</code>	

Ejercicio 2: Fantasma en el Shell

Consideraciones preliminares

Primero, veamos qué hacen los comandos:

<code>ls -al</code>	Lista los archivos del directorio actual.
<code>wc -l</code>	Cuenta la cantidad de líneas del input.
<code>cmd1 cmd2</code>	
<code>ls -al wc -l</code>	

Ejercicio 2: Fantasma en el Shell

Consideraciones preliminares

Primero, veamos qué hacen los comandos:

<code>ls -al</code>	Lista los archivos del directorio actual.
<code>wc -l</code>	Cuenta la cantidad de líneas del input.
<code>cmd1 cmd2</code>	Ejecuta <code>cmd1</code> , y usa su output como input para ejecutar <code>cmd2</code> .
<code>ls -al wc -l</code>	

Ejercicio 2: Fantasma en el Shell

Consideraciones preliminares

Primero, veamos qué hacen los comandos:

<code>ls -al</code>	Lista los archivos del directorio actual.
<code>wc -l</code>	Cuenta la cantidad de líneas del input.
<code>cmd1 cmd2</code>	Ejecuta <code>cmd1</code> , y usa su output como input para ejecutar <code>cmd2</code> .
<code>ls -al wc -l</code>	El comando completo cuenta la cantidad de archivos en el directorio actual.

Ejercicio 2: Fantasma en el Shell

Pseudocódigo

Pasos a seguir...

- Creamos el pipe.
- Creamos los subprocessos.
- Queremos que todo lo que el subprocesso 1 **escriba a stdout**, el subprocesso 2 lo **lea desde stdin**. Para ello, usando dup2:
 - Conectamos **el descriptor de stdout** del subprocesso 1 al **extremo de escritura** del pipe.
 - Conectamos **el descriptor de stdin** del subprocesso 2 al **extremo de lectura** del pipe.
- Ejecutamos el comando `ls -al` en el subprocesso 1.
- Ejecutamos el comando `wc -l` en el subprocesso 2.

Ejercicio 2: Fantasma en el Shell

Detectando el final del input

Todavía falta algo...

El subprocesso 2 tiene que poder **detectar cuando el subprocesso 1 terminó de escribir**. ¿Por qué en este caso es necesario esto?

Ejercicio 2: Fantasma en el Shell

Detectando el final del input

Todavía falta algo...

El subprocesso 2 tiene que poder **detectar cuando el subprocesso 1 terminó de escribir**. ¿Por qué en este caso es necesario esto?

Y, más importante, ¿cómo lo logramos?

Ejercicio 2: Fantasma en el Shell

Detectando el final del input

Todavía falta algo...

El subprocesso 2 tiene que poder **detectar cuando el subprocesso 1 terminó de escribir**. ¿Por qué en este caso es necesario esto?

Y, más importante, ¿cómo lo logramos?

¡Haciendo un buen uso de los pipes!

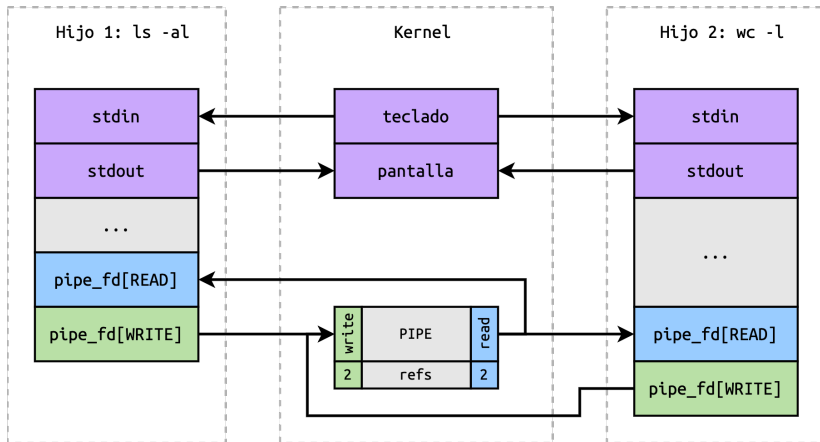
Debemos cerrar los descriptores no utilizados en cada subprocesso.

- El subprocesso 1 jamás va a usar el extremo de lectura del pipe.
- El subprocesso 2 jamás va a usar el extremo de escritura.

Cerrándolos previamente, nos aseguramos de que cuando el subprocesso 1 termine de escribir a *"su stdout"* (que en realidad es el extremo de escritura del pipe), el sistema operativo detecte que **ya no existen más referencias hacia ese descriptor**, retornando un valor de **End-Of-File** cuando el subprocesso 2 intente leer *"su stdin"* (que en realidad es el extremo de lectura del pipe).

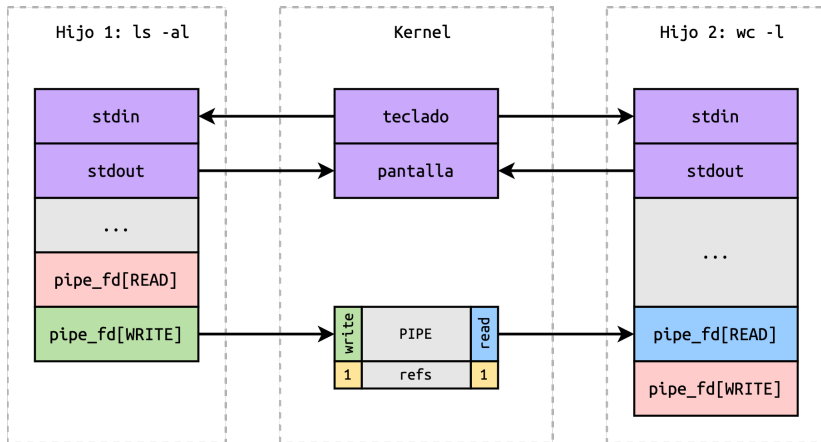
Ejercicio 2: Fantasma en el Shell

Después de los `fork()` los hijos heredan los file descriptors



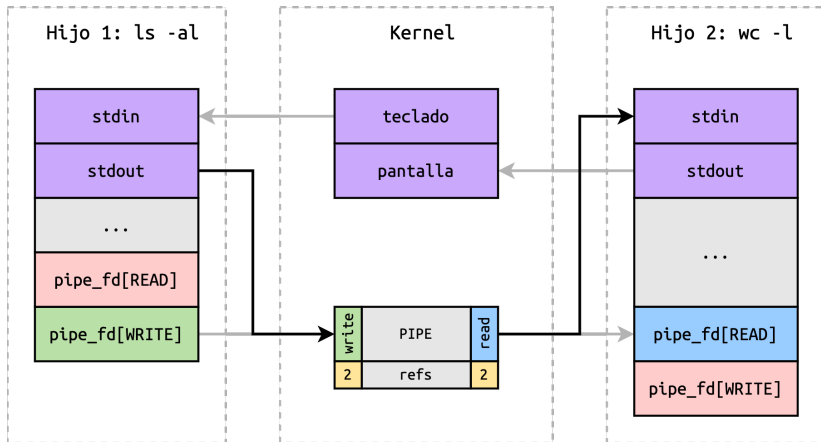
Ejercicio 2: Fantasma en el Shell

Después de cerrar los file descriptors del pipe en cada hijo



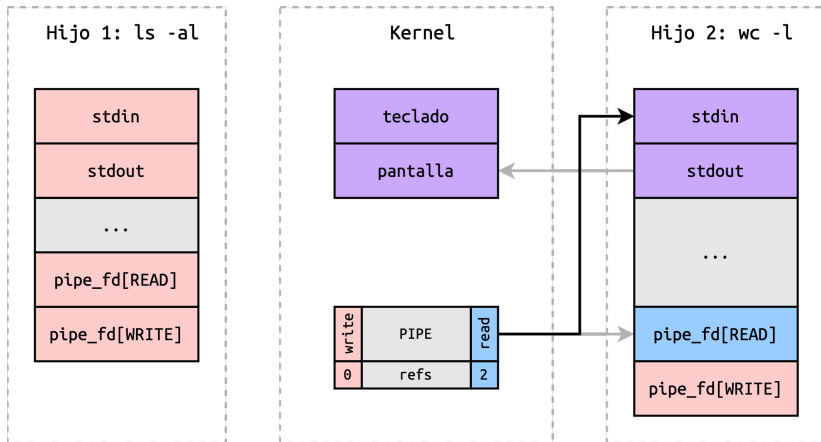
Ejercicio 2: Fantasma en el Shell

Después del dup2



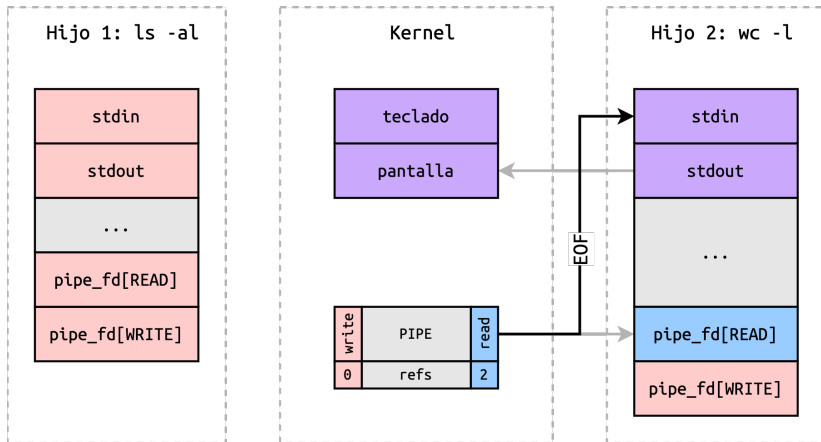
Ejercicio 2: Fantasma en el Shell

Después que termina el hijo 1



Ejercicio 2: Fantasma en el Shell

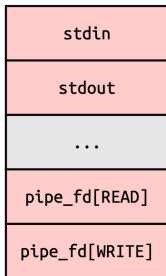
Hijo 2 recibe EOF (End-Of-File)



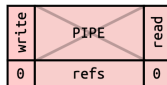
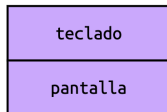
Ejercicio 2: Fantasma en el Shell

Después que termina el hijo 2

Hijo 1: `ls -al`



Kernel



Hijo 2: `wc -l`



Ejercicio 2: Fantasma en el Shell

Resolución

Nos tomamos unos minutos para pensarlo...



...y luego lo resolvemos en pizarrón.

Bonus: Para pensar...

¿Qué tendríamos que **modificar al código** para lograr lo siguiente?

- Encadenar un tercer comando.
 - Por ejemplo: `ls -al | grep e | wc -l`
- Que el output se guarde en un archivo.
 - Por ejemplo: `ls -al | wc -l > file.txt`

Temario

- 1 Repaso
- 2 Ejercicio 1: Los Simonzón
- 3 Ejercicio 2: Fantasma en el Shell
- 4 Ejercicio 3: ¿Cien cabezas piensan mejor que una?
- 5 Sockets
- 6 Conclusión

Ejercicio 3: ¿Cien cabezas piensan mejor que una?

Consigna



Escriba un programa que cuente los números pares en el rango de 2 a mil millones, utilizando para ello una función `bool esPar(long numero)` provista por la cátedra.

El programa dividirá el cálculo de forma equitativa entre varios procesos ejecutándose en paralelo. El número de procesos se especificará como parámetro en la línea de comandos.

Ejercicio 3: ¿Cien cabezas piensan mejor que una?

Consigna (2)

- El proceso primario creará el número especificado de procesos, y luego dividirá el rango de números en subrangos iguales y asignará un subrango a cada proceso secundario.
- Por ejemplo, si la cantidad de procesos p_i es 10, cada subrango tendrá aproximadamente cien millones de números.
 - p_1 calculará en el rango $[2, 100.000.002)$.
 - p_2 calculará en el rango $[100.000.002, 200.000.001)$.
 - ...
 - p_{10} calculará en el rango $[900.000.002, 1.000.000.001)$.
- El proceso primario informará a cada proceso secundario el subrango que le corresponde usando pipes. Luego, los procesos secundarios deberán enviar sus recuentos al primario, también utilizando pipes.
- Finalmente, el primario deberá sumar todos los recuentos e imprimir el resultado total.

Ejercicio 3: ¿Cien cabezas piensan mejor que una?

Pseudocódigo

Proceso primario

- Crear los pipes
- Crear los procesos secundarios
- Calcular e informar los rangos
- Recibir los resultados parciales
- Sumarlos e imprimir el total

Procesos secundarios

- Cerrar pipes
- Recibir los rangos
- Contar los pares
- Informar el resultado

Momento para preguntas

Temario

- 1 Repaso
- 2 Ejercicio 1: Los Simonzón
- 3 Ejercicio 2: Fantasma en el Shell
- 4 Ejercicio 3: ¿Cien cabezas piensan mejor que una?
- 5 Sockets**
- 6 Conclusión

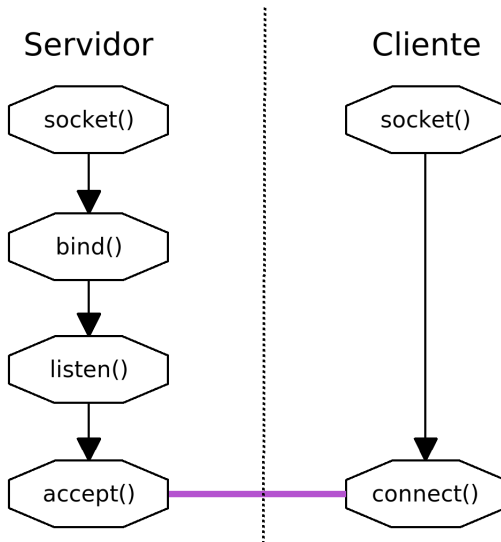
¿Qué pasa si los procesos están en máquinas distintas?

¿Qué pasa si los procesos están en máquinas distintas?

Sockets

- Un **socket** es el extremo de una conexión, constituida por una dirección ip y un puerto.
- Dos procesos que se quieren comunicar entre sí se deben poner de acuerdo usando los sockets correspondientes.
- Los **sockets** pueden trabajar en distintas máquinas.
- Sólo se deben conocer de antemano las direcciones ip y los puertos utilizados para la conexión.
- Normalmente utilizan el paradigma cliente-servidor.

Comunicación Cliente-Servidor usando sockets



¿Tipos de Sockets?

Sockets

- Cuando un socket es creado, el programa tiene que especificar el **dominio de dirección** y el **tipo de socket**.

¿Tipos de Sockets?

Sockets

- Cuando un socket es creado, el programa tiene que especificar el **dominio de dirección** y el **tipo de socket**.
- Hay muchos **dominios de dirección**, pero hay dos mas usados:
 - Unix Domain: Procesos en una maquina local (con lo que vamos a trabajar).
 - Internet Domain: Procesos a traves de Internet.

¿Tipos de Sockets?

Sockets

- Cuando un socket es creado, el programa tiene que especificar el **dominio de dirección** y el **tipo de socket**.
 - Hay muchos **dominios de dirección**, pero hay dos mas usados:
 - Unix Domain: Procesos en una maquina local (con lo que vamos a trabajar).
 - Internet Domain: Procesos a traves de Internet.
 - Hay dos tipos de sockets:
 - Stream Sockets: Tratan la comunicacion como un stream continuo de caracteres.
 - Datagram sockets: Tienen que leer el mensaje entero de una vez.
- ¡Importante!** Dos procesos pueden interactuar entre sí sólo si son sockets del mismo tipo.

- `int socket(int domain, int type, int protocol);`
Crea un nuevo `socket`. Vamos a usar la constante `AF_UNIX` y el tipo `SOCK_STREAM`. El tercer parámetro suele ser 0, indicando al SO que use la configuración predeterminada de protocolos.

Estableciendo sockets: Servidor

- `int bind(int fd, sockaddr* a, socklen_t len);`
Asigna una dirección (nombre o IP y puerto) al `socket`.

```
struct sockaddr_in {  
    short    sin_family; /* must be AF_INET */  
    u_short  sin_port;  
    struct   in_addr sin_addr;  
    char     sin_zero[8]; /* Not used, must be zero */  
}
```

Estableciendo sockets: Servidor

❶ `int listen(int fd, int backlog);`

Setea al socket del servidor como un socket pasivo que recibirá conexiones entrantes. Se maneja una cola para poder recibir varias conexiones entrantes.

❷ `int accept(int fd, sockaddr* a, socklen_t* len);`

Extrae de la cola una solicitud de conexión y establece la comunicación entre los sockets. Se bloquea en caso de no existir conexiones pendientes. Devuelve un nuevo fd para conexión.

Estableciendo sockets: Cliente

- 1 `int socket(int domain, int type, int protocol);`
Crear un nuevo `socket`.
- 2 `int connect(int fd, sockaddr* a, socklen_t* len);`
Conectarse a un `socket` remoto que debe estar escuchando.

Comunicación vía sockets: Mensajes

Una vez que un cliente solicita conexión y ésta es aceptada por el servidor, se puede comenzar el intercambio de mensajes con:

```
ssize_t send(int s, void *buf, size_t len, int flags);  
ssize_t recv(int s, void *buf, size_t len, int flags);
```

Nota

- Por defecto, el envío de mensajes es asincrónico (send se comporta similar al write en pipes).
- A su vez, la recepción con recv se comporta similar al read en pipes. Más detalles en [man 2 send/recv](#).

Veamos un ejemplo en código



¿Cómo hacemos si un servidor se quiere
comunicar con dos clientes a la vez?
Veamos...

Comunicación vía sockets: ¿Bloqueante?

Si usamos llamadas bloqueantes podemos esperar indefinidamente por el primer cliente mientras el segundo tiene mucho que enviarnos y nunca se lo pedimos.

Soluciones:

- 1 Llamadas no bloqueantes con `fcntl(2)`.
- 2 Utilizar un proceso para atender cada cliente (`accept` + `fork`).
- 3 Usar las llamadas al sistema `pselect(2)` o `poll(2)`.

Solución 1: Llamadas No Bloqueantes (`fcntl`)

Idea

Con `fcntl(fd, F_SETFL, O_NONBLOCK)` las llamadas dejan de bloquear: si no hay datos, `recv` devuelve `-1` y setea `errno` en `EAGAIN/EWOULDBLOCK` en vez de dormir al proceso.

Solución 1: Llamadas No Bloqueantes (`fcntl`)

Idea

Con `fcntl(fd, F_SETFL, O_NONBLOCK)` las llamadas dejan de bloquear: si no hay datos, `recv` devuelve `-1` y setea `errno` en `EAGAIN/EWOULDBLOCK` en vez de dormir al proceso.

El problema: espera activa (*busy waiting*)

Para no perdernos ningún cliente tenemos que recorrer **todos** los `fds` en un `while(1)` preguntando “¿tenés algo?”. Aunque nadie envíe nada, el proceso gira al **100 % de CPU** sin hacer trabajo útil.

Solución 1: Llamadas No Bloqueantes (fcntl)

Idea

Con `fcntl(fd, F_SETFL, O_NONBLOCK)` las llamadas dejan de bloquear: si no hay datos, `recv` devuelve `-1` y setea `errno` en `EAGAIN/EWOULDBLOCK` en vez de dormir al proceso.

El problema: espera activa (busy waiting)

Para no perdernos ningún cliente tenemos que recorrer **todos** los `fds` en un `while(1)` preguntando “¿tenés algo?”. Aunque nadie envíe nada, el proceso gira al **100 % de CPU** sin hacer trabajo útil.

Cambiamos “bloquearnos en un cliente e ignorar al resto” por “no bloquearnos nunca pero quemar el procesador”. **Ninguna de las dos sirve**. Necesitamos **concurrency real** o **esperar sobre varios fds a la vez**.

Solución 2: Un proceso por cliente (accept + fork)

Idea

El proceso principal sólo hace accept. Por cada conexión aceptada hace fork: el **hijo** atiende a ese cliente de punta a punta (con llamadas **bloqueantes**, simples), y el **padre** vuelve inmediatamente a esperar la próxima conexión.

Solución 2: Un proceso por cliente (accept + fork)

Idea

El proceso principal sólo hace `accept`. Por cada conexión aceptada hace `fork`: el **hijo** atiende a ese cliente de punta a punta (con llamadas `bloqueantes`, simples), y el **padre** vuelve inmediatamente a esperar la próxima conexión.

- Cada cliente vive en su **propio espacio de direcciones**: no hay `data races` entre clientes, la lógica es secuencial.
- El padre nunca se bloquea atendiendo: sólo reparte conexiones.
- El `fork` **duplica la tabla de descriptores**: hay que cerrar en cada proceso el `fd` que no usa.

accept + fork: el bucle del servidor

```
int sfd = /* socket + bind + listen ya realizados */;

for (;;) {
    int cfd = accept(sfd, NULL, NULL);
    if (cfd == -1) {
        if (errno == EINTR) continue;    /* interrumpido por SIGCHLD */
        perror("accept"); break;
    }

    pid_t pid = fork();
    if (pid == -1) { perror("fork"); close(cfd); continue; }

    if (pid == 0) {                      /* HIJO: atiende a este cliente */
        close(sfd);                      /* no necesita el socket de escucha */
        atender_cliente(cfd);
        close(cfd);
        _exit(0);                        /* _exit, no exit: no toca buffers here */
    }

    close(cfd);                          /* PADRE: no necesita el socket conecta
}
```

¿Y si tenemos 10.000 clientes? (el problema C10K)

Un proceso por conexión no escala

- Cada conexión cuesta un **proceso entero**: memoria, tabla de páginas, tabla de descriptores.
- 10.000 conexiones $\Rightarrow$ 10.000 procesos: el **scheduler** clavado en **cambios de contexto**.
- Compartir estado entre clientes además exige **IPC**.

¿Y si tenemos 10.000 clientes? (el problema C10K)

Un proceso por conexión no escala

- Cada conexión cuesta un **proceso entero**: memoria, tabla de páginas, tabla de descriptores.
- 10.000 conexiones $\Rightarrow$ 10.000 procesos: el **scheduler** clavado en **cambios de contexto**.
- Compartir estado entre clientes además exige **IPC**.

Peor aún: conexiones lentas u ociosas

Con clientes **keep-alive** que conectan y casi no envían, cada uno ocupa un proceso **sin hacer trabajo**. Un puñado de conexiones lentas puede **agotar** todos los procesos.

En el mundo real: Apache (prefork) vs. nginx

Apache — modelo `prefork`

Mantiene un **pool de procesos worker** pre-forkeados (amortiza el costo de fork), pero sigue siendo **1 proceso : 1 conexión activa**. El límite `MaxRequestWorkers` es un techo duro: llenás el pool con clientes lentos y el servidor deja de aceptar.

En el mundo real: Apache (prefork) vs. nginx

Apache — modelo `prefork`

Mantiene un **pool de procesos worker** pre-forkeados (amortiza el costo de fork), pero sigue siendo **1 proceso : 1 conexión activa**. El límite `MaxRequestWorkers` es un techo duro: llenás el pool con clientes lentos y el servidor deja de aceptar.

nginx — modelo `event-driven`

Pocos `workers` (típicamente **uno por core**), cada uno con un **event loop** sobre `epoll/kqueue`, atendiendo **miles** de conexiones concurrentes. La memoria crece con el trabajo, no con la cantidad de conexiones ociosas.

En el mundo real: Apache (prefork) vs. nginx

Apache — modelo `prefork`

Mantiene un **pool de procesos worker** pre-forkeados (amortiza el costo de fork), pero sigue siendo **1 proceso : 1 conexión activa**. El límite `MaxRequestWorkers` es un techo duro: llenás el pool con clientes lentos y el servidor deja de aceptar.

nginx — modelo `event-driven`

Pocos `workers` (típicamente **uno por core**), cada uno con un **event loop** sobre `epoll/kqueue`, atendiendo **miles** de conexiones concurrentes. La memoria crece con el trabajo, no con la cantidad de conexiones ociosas.

Apache terminó adoptando el mismo enfoque con su `event MPM`. Esto es **exactamente** la Solución 3 →

Solución 3: Multiplexar con pselect(2) / poll(2)

La intuición

En vez de **un proceso por conexión**, un **único** proceso le pregunta al kernel: “**dormime hasta que alguno de estos fds tenga algo**”. El kernel lo despierta y le dice **cuál**.

- Sin espera activa (a diferencia de `fcntl`): el proceso **duerme** hasta que hay un evento.
- Sin un proceso por cliente (a diferencia de `fork`): miles de conexiones en **un solo** hilo de ejecución.
- `pselect/poll` escanean toda la lista en cada llamada; `epoll/kqueue` son la versión escalable (lo que usa `nginx` por dentro).

- Beej's Guide to Unix Interprocess Communication. Disponible [aquí](#).
- Beej's Guide to Network Programming: Using Internet Sockets. Disponible [aquí](#).
- Unix socket programming in C. Disponible [aquí](#).
 - Inside NGINX: How We Designed for Performance & Scale. Disponible [aquí](#).
 - The Architecture of Open Source Applications - nginx. Disponible [aquí](#).

Momento para preguntas

Temario

- 1 Repaso
- 2 Ejercicio 1: Los Simonzón
- 3 Ejercicio 2: Fantasma en el Shell
- 4 Ejercicio 3: ¿Cien cabezas piensan mejor que una?
- 5 Sockets
- 6 Conclusión**

Hoy vimos...

- Cómo crear una **topología** de procesos.
- Cómo usar **pipes** para comunicar procesos.
- Cómo usar **dup2** para duplicar un pipe.
- Por qué es importante **usar correctamente** los pipes.
- Para qué sirve la señal de **End-Of-File**.
- Cómo establecer una comunicación bidireccional:
 - Usando un solo pipe por cada par de procesos^a
 - Usando dos pipes por cada par de procesos.

^a ¡muy peligroso!

Hoy vimos...

- Cómo crear una **topología** de procesos.
- Cómo usar **pipes** para comunicar procesos.
- Cómo usar **dup2** para duplicar un pipe.
- Por qué es importante **usar correctamente** los pipes.
- Para qué sirve la señal de **End-Of-File**.
- Cómo establecer una comunicación bidireccional:
 - Usando un solo pipe por cada par de procesos^a
 - Usando dos pipes por cada par de procesos.

^a ¡muy peligroso!

Cómo seguimos...

- Pueden hacer toda la guía 1.
- Próximamente: taller de **IPC**.